

面向 AI 辅助编程的学习者能力观测与动态追踪研究

1. 研究背景与研究现状

1.1 AI 辅助编程对传统编程能力评价提出的新问题

近年来，以大语言模型为基础的代码生成技术快速发展。GitHub Copilot、ChatGPT、Cursor 等 AI 编程工具已经能够参与需求理解、代码生成、程序修改、测试生成和错误诊断等多个软件开发环节。编程活动因此逐渐由传统的“开发者直接完成代码编写”转向“开发者与 AI 共同完成程序设计和实现”。

这种变化并不意味着传统计算机科学知识已经失去作用。针对 Vibe Coding 的最新研究表明，即使开发者主要通过自然语言与生成式 AI 完成程序构建，既有计算机科学知识仍然能够显著预测其任务表现 [12]。这说明 AI 改变了代码产生方式，但并没有完全消除程序理解、算法知识和问题求解能力的重要性。

与此同时，AI 对即时任务表现和长期能力形成的影响并不完全一致。Shen 和 Tamkin 在 2026 年开展的随机实验比较了开发者在使用与不使用 AI 的条件下学习陌生 Python 库的过程。结果显示，AI 辅助并不必然转化为相应的技能形成，部分高度依赖 AI 完成任务的参与者在后续概念理解、代码阅读和调试测试中表现较弱；但保持主动解释、理解和推理活动的参与者能够在使用 AI 的同时维持较好的学习效果 [13]。这一结果表明，问题的关键并不只是“是否使用 AI”，而可能在于学习者在 Human-AI programming process 中采取了怎样的交互方式以及保留了多少主动认知活动。

因此，AI 辅助编程对传统的学习者能力评价提出了一个新的基础问题：**当最终代码越来越多地成为学习者与 AI 共同作用的结果时，代码是否正确、通过多少测试以及最终程序内容，还能够多大程度上被解释为学习者自身知识和能力状态的直接观测？**

这一问题与 Knowledge Tracing (KT) 的基本思想高度相关。KT 的核心任务并不是简单记录一次作答的正确或错误，而是根据学习者连续的历史交互推断其无法被直接观察的知识状态，并预测未来学习表现。因此，当 AI 改变学习者 response 的生成过程以后，原有 KT 中“什么可以作为学习者状态有效观测”的问题也需要重新讨论。

1.2 Knowledge Tracing 的技术发展

早期具有代表性的 Bayesian Knowledge Tracing (BKT) 将学习者对某一知识组件的掌握情况表示为不可直接观测的概率状态，并通过连续作答结果进行更新[1]。BKT 中 Guess 和 Slip 参数的存在已经体现出 KT 的一个基本认识，即一次正确或错误的外在表现并不必然与真实知识掌握完全一致。

2015 年，Piech 等提出 Deep Knowledge Tracing (DKT)，使用循环神经网络对学习者长期交互序列进行建模，使知识状态不再完全依赖人工设定的概率转移，而由神经网络从交互历史中学习潜在状态表示[2]。随后，研究重点逐渐转向如何更有效地利用长期历史信息。SAKT 使用 Self-Attention 根据当前问题选择更加相关的历史学习行为[3]；AKT 则进一步结合问题难度、时间距离和 Rasch 模型等认知先验，使注意力机制更加符合实际学习规律[4]。

从方法发展角度看，这一阶段的演进主要发生在知识状态更新模型一侧，可以概括为由显式概率状态向深度序列状态，再向基于 Attention 的选择性历史建模发展。与此同时，学习者一次交互的基本表示仍主要由题目、知识组件以及 correct/incorrect 等结果构成。

1.3 Programming Knowledge Tracing: 由模型结构创新向学习证据扩展

开放式编程任务与传统选择题存在明显区别。学生提交的程序除了最终正确性以外，还包含代码结构、算法选择、变量使用、错误类型以及连续修改过程。因此，近年来 Programming Knowledge Tracing (PKT) 的一个重要发展趋势，是不断扩大能够作为学习者状态观测的证据来源。

Code-DKT 在 2022 年将学生实际提交的源代码引入 Deep Knowledge Tracing，通过代码特征和 Attention 机制扩展传统 DKT。实验结果显示，在其研究数据上，Code-DKT 相比 DKT 获得了约 3.07% 至 4.00% 的 AUC 提升，说明学生代码本身包含传统 correct/incorrect 标签没有充分利用的知识信息 [5]。

同年，Liu 等在 EMNLP 提出了 Open-ended Knowledge Tracing (OKT)。该工作进一步指出，只预测下一道题正确与否仍然会丢失开放式编程响应中的大量信息，因此将研究目标扩展为直接预测学生下一次可能产生的完整代码。其方法将语言模型中的程序生成能力与学生知识追踪结合，使 KT 不再局限于二值 response prediction [6]。

TIKTOC 在 LAK 2025 中进一步将整道程序的 correctness 细化到 test-case level。该方法同时分析学生代码和每个测试用例的通过情况，并利用多任务学习同时预测下一次代码以及测试用例结果，使学习者状态能够反映比整体程序分数更加细粒度的行为信息 [7]。其官方研究说明明确将该工作定义为同时预测开放式学生代码与各测试用例结果的 Granular Knowledge Tracing 框架。

ACL 2025 的 SQKT 则进一步将学生主动提出的问题作为 Programming KT 的新观测来源。作者认为，源代码能够反映学生产生了什么结果，但学生在编程过程中提出的问题可能更加直接地暴露其困惑和知识缺口。因此，SQKT 同时编码题目、源代码、学生问题以及从问题中自动抽取的技能信息，再预测学生后续编程表现。其结果表明，问题的语义内容本身能够在已有代码信息之外提供额外预测信号 [8]。

Coda 在 KDD 2025 中又从长期代码轨迹中的噪声问题出发，指出学生长期提交记录中可能包含与真实能力更新关系较弱的无关提交以及细微修改。该方法通过 Code Graph 与 GCN 识别和抑制噪声，再以 model-agnostic adaptor 的方式增强已有 PKT 模型，而不是重新设计一个完整的 KT 主干网络 [9]。这一工作说明，对于新的学习证据或新的数据问题，可以通过轻量级适配机制扩展已有 PKT，而不一定需要完全重构模型。

与此同时，大语言模型开始越来越多地参与 Programming KT。2026 年 ACL Findings 的 KCGen-KT 使用 LLM 自动生成和标注开放式编程题的 Knowledge Components，并进一步构建 LLM-based KT 模型利用这些细粒度知识组件预测学生未来表现。该工作在两个真实学生代码数据集上进行了定量、定性和教师人工评价，说明 LLM 已经能够参与 KC 发现、问题标注及知识状态建模等多个环节 [10]。

因此，Programming KT 近年来的发展可以概括为学习证据逐渐由 **Correctness** 扩展到 **Code Content**、**Open-ended Response**、**Test-case Behavior**、**Student Questions**、**Knowledge Components** 以及 **Long-term Code Process**。这一发展路线表明，PKT 的核心问题已经不仅是采用何种 RNN 或 Transformer，而是不断重新回答：**哪些行为信息能够更加有效地反映学习者状态？**

1.4 AI-assisted programming 与 learner modeling 的最新发展

尽管 Programming KT 已经不断扩大输入信息，但 AI 辅助编程又引入了一类新的交互过程。需要明确的是，当前已经不能笼统地说“现有 KT 没有考虑 AI”。事实上，LLM 已经被广泛用于 KT 的知识抽取、代码生成和状态估计；AI tutor 与 KT 的结合也已经出现。

例如，Scarlato 等提出 Dialogue Knowledge Tracing，使用 LLM 自动识别 tutor-student dialogue 中的知识组件和学生回答正确性，再在完整对话序列中进行 Knowledge Tracing；进一步提出的 LLMKT 则直接利用开放式对话文本进行学生状态预测 [11]。这表明 conversational interaction 本身已经能够被用于 KT，而不必局限于传统题目正确率。

另一方面，AI-assisted programming 行为本身也已成为独立研究对象。RECAP 是 ACL 2026 System Demonstrations 发表的 AI 编程交互采集平台，可以在 VS Code 中同步记录 AI chat、Prompt、AI 建议及细粒度代码编辑，再重建完整时间线。该系统已经在真实大学软件工程课程中采集 41 名学生的 2034 条 Prompt 和 8239 次代码编辑，为 Human-AI programming trajectory 的系统分析提供了技术基础 [14]。

Shao 等在 2026 年进一步分析了 163 名 Python 初学者与 LLM 的完整聊天记录和最终代码，识别出从直接委托到逐步 refinement 的多种 Prompt-level trajectory，并将这些行为轨迹与作业结果和课程表现进行比较。其结果说明，学习者与 AI 的交互方式本身可以成为理解其 self-regulation 和 learning orientation 的窗口 [15]。

CHI 2026 的 ClassAid 则将 AI 进一步嵌入真实编程课堂。该系统持续监控学生与 AI 的交互、代码执行和修改过程，并由智能 TA Agent 根据学生状态动态提供直接解决方案、启发式提示或保持静默。ClassAid 的研究重点是课堂中的实时人机教学协调，其部署实验覆盖 54 名学生，同时也说明 Human-AI programming interaction 已经可以被持续记录并用于实时学习支持 [16]。

此外，CoTutor 已经明确将 Knowledge Tracing 与 GenAI personalized education 结合。其核心使用增强的 Bayesian Knowledge Tracing 对学生状态进行持续估计，再利用 GenAI 进行自适应反馈和资源推荐 [17]。这说明“AI + KT”本身已不是研究空白；AI 在此类工作中主要作为 tutor 或 intervention mechanism，而学习者代码和判题结果仍然构成主要状态观测。

CogSci 2026 已接收的研究还进一步探索 Structured AI Guidance 与 Graph-enhanced Interactive Knowledge Tracing 的结合，说明 AI scaffolding、interaction trace 和 knowledge-state estimation 已经开始在同一教育系统内出现 [18]。

综合以上研究可以发现，当前已经分别形成了三条较成熟的发展线：一是 Programming KT 不断增加代码、测试、问题和 KC 等 learner evidence；二是 AI-assisted programming 开始系统记录 Prompt、AI Response 和代码修改轨迹；三是 GenAI 与 KT 开始结合用于知识状态估计和个性化干预。但是，当 coding AI **直接参与学生最终代码响应的产生过程**以后，如何重新解释传统 PKT 中的 response evidence，以及如何将 Human-AI interaction 本身作为正式的纵向学习者观测用于 Programming KT，目前仍处于较早阶段。

2. 选题依据与可行性分析

本课题的基本出发点是，AI 辅助编程正在改变 Programming Knowledge Tracing 中“学生响应”的含义。在传统编程环境下，可以近似认为学生提交的代码主要由其当前知识和能力状态产生，因此程序正确性、代码内容及测试结果能够作为 learner state 的重要观测。然而，在 AI-assisted programming 环境中，最终代码可能受到学生状态、AI 模型能力以及学生与 AI 交互方式的共同影响。相同的正确代码可能来自完全不同的 Human-AI production process，这使最终 response 与 learner state 之间原本较直接的关系变得更加复杂。

从理论上讲，这一问题与 KT 的隐状态思想具有较好的兼容性。传统 Programming KT 可以粗略表示为：

$$O_t \sim p(O_t | K_t, Q_t)$$

其中，(K_t) 表示学习者在时间 (t) 的潜在知识状态，(Q_t) 表示当前问题，(O_t) 表示观察到的正确性、程序代码或测试结果。

当 AI coding assistant 直接参与响应生成以后，更合理的观测过程可以表示为：

$$O_t \sim p(O_t | K_t, Q_t, A_t, I_t)$$

其中, (A_t) 表示当前 AI assistance, (I_t) 表示学生与 AI 的交互过程。该形式并非要求研究必须建立完整的概率生成模型, 而是用于说明: AI 的直接参与增加了影响程序 response 的外部因素, 因此仅根据最终 response 反推 learner state 可能出现信息损失。

在研究方法上, 本课题具有较好的现有基础。Code-DKT、OKT、TIKTOC、SQKT 和 KCGen-KT 已经分别证明代码内容、开放式响应、测试用例、学生问题和细粒度 KC 可以被有效纳入 Programming KT[5-10]。特别是 SQKT 的研究逻辑与本课题具有较强相似性: 该工作并没有简单设计更加复杂的 KT backbone, 而是首先证明“学生问题”包含代码本身没有充分反映的增量信息, 再进一步设计模型使用这一新的 observation channel[8]。本研究可以沿用类似的研究逻辑, 将 Student-AI interaction 作为新的候选 learner evidence。

在工程实施方面, AI 编程交互数据的采集也已有现实基础。RECAP 已经证明 Prompt、AI Response、代码编辑和时间关系可以在不显著干扰真实开发流程的情况下进行同步记录[14]; ClassAid 也证明学生与 AI 的代码修改和交互历史可以在课堂环境中实时采集[16]。因此, 本研究无需从零证明 Human-AI programming trajectory 是否能够被记录, 而可以将主要精力集中在这些数据能否改进 learner state estimation。

从模型实现角度, Coda 的 model-agnostic adaptor 为控制工作量提供了较好的参考[9]。如果本研究首先发现 AI interaction 确实具有额外预测价值, 可以在 DKT、AKT 或现有 Programming KT backbone 前增加 AI-aware observation adaptor, 而不必立即开发完整的新型 Transformer。只有在实验表明已有单一隐状态无法有效解释新观测时, 再进一步发展结构化状态模型。

因此, 本研究的主要挑战不是现有基础技术是否可实现, 而是两个更加核心的问题: 其一, AI interaction 在控制代码、正确性和测试结果以后是否仍包含关于学习者未来状态的独立信息; 其二, 这类信息应该如何编码并纳入长期动态状态更新。前者决定研究假设是否成立, 后者则决定最终模型创新的深度。

3. 研究空白与核心问题

综合现有文献, 可以看到 Programming KT、AI-assisted programming behavior 和 AI-supported learner modeling 三个方向已经分别形成了较为丰富的研究成果, 因此本研究不能将创新表述为“首次将 AI 应用于 KT”, 也不能将单纯采集学生 Prompt 或代码修改记录作为主要贡献。

现有 Programming KT 已经能够利用题目、正确性、代码内容、测试用例、学生问题、KC 和代码修改等多种信息估计学习者状态; 现有 AI-assisted programming 研究也已经能够持续记录 Prompt、AI Response、代码采纳和修改等行为; 部分最新系统则进一步将 AI 辅导与 KT 状态估计结合起来。然而, 从目前检索到的同行评审研究看, 将以下三个环节完整结合的工作仍然较少: 第一, AI coding assistant 直接参与学生编程响应生成; 第二, 将 Human-AI interaction trajectory 作为正式的 Programming KT observation; 第三, 利用这些信息进行跨任务、纵向的 learner knowledge 或 competence state estimation。

因此, 本研究拟解决的核心问题可以表述为:

在 AI-assisted programming 环境中, 当题目、学生最终代码、正确性和测试结果等传统 Programming KT 信息已经已知时, Human-AI interaction 是否仍然包含能够预测学习者未来表现的增量信息; 如果存在, 应如何将这类信息表示并纳入动态 Programming Knowledge Tracing?

这一问题进一步包含三个逐级展开的研究问题。

首先, 需要判断传统 PKT evidence 在 AI-assisted programming 下是否仍然充分。如果学习者与 AI 的交互过程在控制最终代码和测试结果后几乎不能提供额外信息, 则说明传统 Programming KT 仍然具有较强适用性, 也没有必要人为增加复杂模型。

其次，如果 AI interaction 确实具有增量信息，需要判断“是否使用 AI”“使用了多少 AI”与“如何使用 AI”之间是否存在本质差异。简单的调用次数、Prompt 数量和时长可能只能反映 AI 使用强度，而 Prompt 内容、AI Response、采纳行为、人工修改和验证过程可能更加接近真实的认知参与。

最后，需要判断这些新增观测是否只是提高下一题 correctness prediction，还是能够进一步帮助模型估计代码验证、错误识别或调试等能力。如果后者成立，则说明 AI-assisted programming 环境下有必要进一步扩展传统 Programming KT 的 learner state representation。

4. 拟创新点

4.1 面向 AI 参与代码生成过程的 Programming KT 观测模型扩展

现有 Programming Knowledge Tracing 的主要观测通常由题目、源代码、正确性、测试结果和知识组件构成。这些信息能够有效描述学生产生了什么程序以及程序最终表现如何，但在 AI-assisted programming 环境中，最终程序可能是 Human 与 AI 的联合产物。

本研究拟将传统 Learner-Problem Interaction 扩展为 Learner-AI-Problem Interaction，将一次编程任务中的 Prompt、AI Response、生成代码的采纳或拒绝、人工修改、测试与验证等信息纳入学习者状态观测过程。

该创新的重点不在于简单增加一组 feature，而在于研究 AI 参与 response generation 以后，Programming KT 是否需要由单纯分析最终程序结果进一步扩展到分析程序的生成过程。如果 Human-AI interaction 在传统代码信息之外具有稳定的增量预测能力，则可以说明 AI-assisted programming 环境确实引入了一类新的有效 learner evidence。

4.2 Human-AI Programming Interaction 的结构化表示

AI 使用过程具有明显异质性。同样是使用生成式 AI，一名学习者可能直接要求生成完整答案并提交，另一名学习者则可能先询问概念、逐步生成、修改错误并通过测试不断验证。因此，单纯使用 Prompt 数量、AI 调用次数或交互时长可能无法准确表示学习者在 AI-assisted programming 中的实际行为。

本研究拟从行为和语义两个层面对 Human-AI interaction 进行表示。行为层面主要描述直接委托、解释请求、代码采纳、部分修改、拒绝生成结果、重新生成、测试和验证等交互事件；语义层面则利用文本或代码编码模型表示 Prompt、AI Response 以及其与最终代码之间的关系。

通过分别比较简单统计信息、行为表示和完整交互语义，可以进一步判断 AI 使用强度与 AI 使用方式何者具有更强的 learner-state information，并验证完整交互轨迹是否在传统 Programming KT evidence 之外具有独立价值。

4.3 面向 AI-assisted programming 的动态能力追踪方法

在验证新增观测信息有效以后，本研究拟进一步探索 AI-aware Programming Knowledge Tracing 方法。研究初期优先采用相对稳健的扩展方式，在已有 DKT、AKT 或 Programming KT backbone 基础上增加 AI-aware observation module，通过新的 interaction encoder 和 fusion/adaptor 将 Human-AI information 纳入状态更新。

这种设计可以将“新观测信息是否有效”和“新网络结构是否有效”两个问题区分开。如果同一 AI interaction representation 在多个不同 KT backbone 上均能够稳定改善预测，则可以更加有力地说明性能增益来源于新的 learner evidence，而不是网络参数量增加。

如果后续实验进一步表明传统单一隐状态无法同时解释任务表现和代码验证、错误识别等能力，则可以在此基础上继续发展 structured competence tracing model，对不同能力状态进行显式建模。因此，本研究的方法创新可以根据实际实验结果由轻量 Observation Adapter 逐步扩展到更加完整的新型 KT 架构，在控制研究工作量的同时保留进一步的方法创新空间。

5. 拟采用的技术路线与实验设计

5.1 学习交互表示

对于学习者在时间 (t) 完成的一次 AI-assisted programming interaction, 可将该次学习记录表示为:

$$X_t = \{Q_t, C_t, R_t, A_t, V_t\}$$

其中, (Q_t) 表示编程题目及其相关知识组件, (C_t) 表示学习者产生的阶段性或最终代码, (R_t) 表示正确性及测试结果, (A_t) 表示 Human-AI interaction trajectory, (V_t) 表示代码验证、错误判断或调试相关证据。

其中前三类信息基本对应现有 Programming KT 的传统观测, 而 (A_t) 与 (V_t) 构成本研究重点探索的新增信息。

5.2 多源信息编码

题目文本和知识组件可以采用文本编码模型形成语义表示; 代码可以参考 Code-DKT、OKT 和 TIKTOC, 根据数据规模采用 ASTNN、CodeBERT、CodeT5 或其他代码表示方法; 测试结果可进一步细分为 public test、hidden test 以及错误类型等结构化信息。

Human-AI interaction 将分别建立行为表示和语义表示。Prompt 和 AI Response 可以采用语言模型进行语义编码, 代码采纳、人工修改、测试、重新生成和验证等操作则作为时间序列行为特征。不同模态经过独立编码后, 通过线性投影、Attention 或 lightweight fusion adaptor 映射到统一表示空间。

这一部分可以重点参考 SQKT 的异构信息融合方法, 将原有 Student Question channel 扩展为更加复杂的 Student-AI Interaction channel; 同时借鉴 Coda 的 model-agnostic adaptor 思路, 使新增模块能够挂接到不同的 KT backbone 上。

5.3 动态状态更新

初始阶段可以使用成熟 KT 模型完成动态状态更新:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, X_t)$$

其中, ($\mathbf{h}_t$) 表示第 (t) 次编程交互后的学习者状态, ($f(\cdot)$) 可以采用 LSTM/DKT、AKT 或其他 Programming KT 模型。

研究初期不预先假定状态必须被拆分为多个独立维度, 而首先验证新增 AI interaction information 能否稳定改善未来表现估计。如果后续结果表明任务表现与代码验证能力之间存在明显不同步, 再进一步考虑将状态表示结构化。

5.4 多任务预测

参考 TIKTOC 同时预测程序代码和 test-case behavior 的思想, 本研究拟避免模型最终仍然仅围绕传统 correctness 训练, 而设置至少两类预测目标。

第一类为 Task Performance Prediction, 用于预测下一次程序完成情况、测试用例通过情况或 hidden-test performance。第二类为 Verification/Debugging Prediction, 用于预测学习者能否判断潜在错误、识别边界问题或完成相应调试任务。

联合训练目标可初步表示为:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{task} + \lambda_2 \mathcal{L}_{verify}$$

其中, $\mathcal{L}_{task}$ 与 $\mathcal{L}_{verify}$ 分别对应任务表现和验证能力预测损失, 具体权重在实验中确定。

AI 与 No-AI 条件的比例以及 verification task 的具体设计将在后续实验阶段进一步确定。无 AI 条件主要作为隔离 AI 直接提供答案影响的实验控制方式, 而不是将完全脱离 AI 完成编程视为最终能力评价标准。

5.5 对照与消融实验

实验首先需要建立一组逐步增加信息的基线。传统 KT baseline 仅使用 Problem、KC 和 Correctness; 随后加入代码形成 Code-aware KT; 进一步加入 test-case 或细粒度 KC 形成较完整的 Programming KT baseline。在此基础上, 再增加 AI 调用次数、Prompt 数量等简单统计信息, 最后加入完整的 AI interaction behavior 与 semantic representation。

这一比较可以依次回答三个问题: 单纯知道学生使用了 AI 是否具有价值; AI 使用模式是否具有额外价值; 完整 Human-AI interaction 是否包含传统代码和测试信息中没有的 learner-state information。

同时需要对 Prompt semantics、AI Response、代码采纳、人工修改、测试和验证等信息分别进行消融, 以分析各类信号的实际贡献。

研究尤其关注一种控制当前任务表现后的评价方式。例如, 两名学习者当前任务均能够通过全部公开测试, 但 Human-AI interaction trajectory 明显不同。如果模型能够利用这些轨迹预测二者在未来 hidden-test、错误识别或 debugging task 上的差异, 则可以更有力地证明 AI interaction 不是简单的附加日志, 而是传统 PKT 中缺失的有效学习者观测。

6. 数据来源与数据集选取

本研究所需数据大致包含两个层面。第一类是传统 Programming Knowledge Tracing 数据, 用于复现现有模型、建立 Code-aware baseline, 并研究代码、测试和 KC 等已有学习证据。第二类是包含真实 Human-AI interaction 的编程数据, 用于验证本研究提出的新增观测信息。

CodeWorkout 是当前 Programming KT 领域具有代表性的公开数据来源之一, 包含真实学生完成开放式编程题时的代码提交与学习记录, 并被 OKT、TIKTOC 以及 KCGen-KT 等研究使用或进一步处理[6-7,10]。因此, 该类数据适合用于 DKT、Code-DKT、OKT 或 TIKTOC 等基础模型复现, 以及 Code/KC/Test-level evidence 的前期分析。TIKTOC 已进一步将 CodeWorkout 中部分题目扩展为具有 test-case-level correctness 的数据, 并公开了相应处理数据及模型实现。

然而, 传统 Programming KT 数据主要产生于没有系统记录 coding AI interaction 的环境, 因此无法单独支撑本研究关于 AI-mediated response generation 的核心问题。针对这一部分, 一种方案是继续调研是否存在能够满足研究需求的公开 Human-AI coding dataset; 另一种方案是在后续实验中自行采集一定规模的数据。

RECAP 已经为后一种方案提供了较成熟的工程参考。该平台能够同步记录 Prompt、AI Response、代码编辑和时间关系, 因此未来可以直接借鉴其数据采集设计, 而无需重新设计完整的 AI coding logger。

因此, 本研究目前的数据策略可以概括为: 利用公开 Programming KT 数据完成基础技术验证和 baseline 建立, 再利用现有或自建 AI-assisted programming interaction 数据验证新增 learner evidence。最终数据集的具体组合将在完成公开数据结构分析和 AI interaction dataset 调研后确定。

7. 拟采用的评价指标与预期结果

对于传统 Programming KT 的未来表现预测，可继续采用 Accuracy、F1-score、AUROC 和 Log Loss 等常见指标。对于 test-case-level 或 hidden-test prediction，也可以采用 AUROC、AUPRC 和 F1 等指标评价模型在更加细粒度程序结果上的表现。

对于 Verification/Debugging Prediction，可根据实验任务采用错误识别准确率、Precision、Recall、F1-score、调试成功率以及完成时间等指标。

本研究最重要的评价并不是单纯比较完整模型是否获得更高 AUC，而是检验新增信息的 **incremental predictive validity**。具体而言，需要比较在传统 Problem、Code、Correctness 和 Test Evidence 已知后，加入 AI interaction information 是否仍然能够显著提高未来任务表现和验证能力预测。

预期结果可分为三个层次。首先，希望验证不同 Human-AI interaction pattern 在传统程序表现相同的情况下仍可能对应不同的未来学习表现。其次，希望证明包含 Prompt、AI Response、代码修改和验证行为的完整 interaction representation 相比简单的 AI 调用次数等统计特征具有更强预测能力。最后，如果这些信息能够稳定改善多个 KT backbone，则进一步探索 AI-aware Programming KT 或更加结构化的 competence tracing model。

研究并不预设复杂新模型一定优于已有 KT。如果最终实验发现 AI interaction 在控制代码和测试结果后贡献很低，这一结果本身也能够说明传统 Programming KT 的程序观测可能比预期更加稳健。因此，研究设计不依赖某一种预设结论成立。

8. Discussion：潜在问题与研究风险

当前研究方向虽然具有较明确的现实问题和技术发展基础，但仍存在若干需要在正式实施前重点解决的问题。

首先，研究创新最终能否成立，取决于 AI interaction 是否真正提供传统 Programming KT evidence 未包含的信息。如果完整 Human-AI trajectory 最终只能带来极小的预测提升，则模型很容易被解释为简单增加特征，而不能形成较强的方法贡献。因此，后续实验必须重点验证在 Code、Correctness 和 Test information 已知的情况下，AI interaction 是否具有独立的增量预测价值，而不能只比较最终模型的总体 AUC。

其次，Human-AI interaction 的表示粒度需要控制。如果仅使用 Prompt embedding，可能无法充分反映学习者对 AI 输出的采纳、验证和修改过程；如果对全部行为进行人工精细标注，又会导致数据标注成本过高。已有 Prompt trajectory、RECAP 和 ClassAid 等工作可以为行为 taxonomy 提供参考，但仍需要结合本研究的 KT 目标确定更加适合动态建模的粒度。

第三，数据规模可能是整个研究最主要的实际约束。传统 Programming KT 公共数据通常具有较长的学生交互序列，但缺少 AI log；AI-assisted programming 数据则往往记录更加丰富，但学生人数和连续任务数量相对有限。KT 模型尤其是大型 Attention 模型需要足够长的交互序列，因此最终模型复杂度应根据实际数据规模决定。如果数据量有限，轻量级 LSTM、状态空间模型或 adaptor-based 方法可能比复杂 Transformer 更合理。

第四，本研究需要合理控制总体工作量。Programming KT、代码表示、Human-AI interaction、行为实验和动态建模本身都可以形成独立研究方向。如果同时重新设计代码编码器、AI interaction taxonomy、KT backbone 和完整实验平台，工作量会迅速扩大。因此更合理的实施顺序是首先使用成熟代码编码与 KT backbone，将主要创新集中在 observation model 和 interaction evidence；在核心假设得到支持以后，再决定是否进一步发展新的 structured KT architecture。

第五，需要持续关注快速发展的相关研究。2026 年已经出现 RECAP、ClassAid、CoTutor 以及 Structured AI Guidance + Knowledge Tracing 等与本研究相邻的工作，说明 AI-assisted programming learner modeling 正在迅速形成新的研究方向。因此，在研究过程中需要持续跟踪 ICSE、FSE、CHI、KDD、ACL、LAK、EDM 等会议的新论文，并在正式投稿前重新确认研究空白。

总体而言，本研究当前最核心的技术与理论问题可以概括为：**AI coding assistant 的直接参与是否改变了 Programming Knowledge Tracing 中“学习者响应”的观测含义，以及 Human-AI interaction 是否能够成为传统代码与正确性信息之外具有独立价值的动态能力证据。**

如果这一假设得到实证支持，则后续可以进一步发展面向 AI-assisted programming 的新型 Programming Knowledge Tracing 方法；如果传统证据已经能够充分解释未来能力，则同样可以帮助界定 AI 时代 Programming KT 现有范式的适用边界。

参考文献

- [1] CORBETT A T, ANDERSON J R. Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge[J]. *User Modeling and User-Adapted Interaction*, 1995, 4(4): 253–278.
- [2] PIECH C, BASSEN J, HUANG J, et al. Deep Knowledge Tracing[C]//Advances in Neural Information Processing Systems. 2015, 28.
- [3] PANDEY S, KARYPIS G. A Self-Attentive Model for Knowledge Tracing[C]//Proceedings of the 12th International Conference on Educational Data Mining. 2019: 384–389.
- [4] GHOSH A, HEFFERNAN N, LAN A S. Context-Aware Attentive Knowledge Tracing[C]//Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2020.
- [5] SHI Y, CHI M, BARNES T, PRICE T W. Code-DKT: A Code-based Knowledge Tracing Model for Programming Tasks[C]//Proceedings of the 15th International Conference on Educational Data Mining. 2022: 50–61.
- [6] LIU N, WANG Z, BARANIUK R, LAN A. Open-ended Knowledge Tracing for Computer Science Education[C]//Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing. 2022: 3849–3862.
- [7] DUAN Z, FERNANDEZ N, HICKS A, LAN A. Test Case-Informed Knowledge Tracing for Open-ended Coding Tasks[C]//Proceedings of the 15th International Learning Analytics and Knowledge Conference. 2025: 238–248.
- [8] KIM D, KIM S, JO Y. Knowledge Tracing in Programming Education Integrating Students' Questions[C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. 2025: 27703–27718.
- [9] GAO W, LIU Q, LI R, et al. Denoising Programming Knowledge Tracing with a Code Graph-based Tuning Adaptor[C]//Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2025.
- [10] DUAN Z, FERNANDEZ N, LEKSHMI NARAYANAN A B, et al. Automated Knowledge Component Generation and Interpretable Knowledge Tracing in Coding Problems[C]//Findings of the Association for Computational Linguistics: ACL 2026. 2026: 33405–33423.
- [11] SCARLATOS A, BAKER R S, LAN A. Exploring Knowledge Tracing in Tutor-Student Dialogues using LLMs[C]//Proceedings of the 15th International Learning Analytics and Knowledge Conference. 2025.

- [12] THORGEIRSSON S, WEIDMANN T B, SU Z. Computer Science Achievement and Writing Skills Predict Vibe Coding Proficiency[EB/OL]. 2026.
- [13] SHEN J H, TAMKIN A. How AI Impacts Skill Formation[EB/OL]. arXiv:2601.20245, 2026.
- [14] HE K, MA Q, CHEN V, CHI W, WU T. RECAP: An End-to-End Platform for Capturing, Replaying, and Analyzing AI-Assisted Programming Interactions[C]//Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. 2026.
- [15] SHAO T, FEIJÓO-GARCÍA M, ZHANG Y, et al. Tracing Prompt-Level Trajectories to Understand Student Learning with AI in Programming Education[EB/OL]. 2026.
- [16] ZHANG G, SUN G, XIA M, LIANG R. ClassAid: A Real-time Instructor-AI-Student Orchestration System for Classroom Programming Activities[C]//Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. 2026.
- [17] WANG Y, YU P D, TAN C W. Future-Proofing Programmers: Optimal Knowledge Tracing for AI-Assisted Personalized Education[J]. *IEEE Signal Processing Magazine*, 2026.
- [18] ZHAO H, WU Y, LU Z, CHEN L. Free or Systematic? Structured AI Guidance Improves Tutor Knowledge-State Estimation Quality and Self-Regulated Learning in Programming Education[C]//Proceedings of the Annual Meeting of the Cognitive Science Society. 2026.